

Ejercicio de partición y asignación de roles

1. ESCENARIO GENERAL: Instituto Médico

Enunciado: Un instituto médico privado desea organizar su base de datos de pacientes, profesionales y actividades médicas de forma que:

1. **Proteja la información sensible de los pacientes**, separándola de los datos operativos.
2. **Registre automáticamente cualquier cambio realizado sobre los datos personales sensibles** (como nombre o correo electrónico).
3. **Controle el acceso a la información según el perfil del usuario:** recepcionista, profesional o administrador.
4. **Gestione los turnos otorgados a los pacientes para diferentes actividades** del instituto

Objetivo: diseñar una base de datos en MySQL que

- Aplique **partición vertical** para dividir los datos personales de los pacientes de los datos administrativos.
- Utilice **triggers** para registrar los cambios sensibles en una tabla de auditoría (LogCambios).
- Defina y asigne **usuarios con distintos niveles de acceso** según su rol en la institución.
- Gestione los turnos otorgados, permitiendo que ciertos roles los registren.

Generaremos:

- 1) Una base de datos InstitutoMedico.
- 2) Tablas para:
 - PacientesDatos: contiene datos personales (DNI, nombre, dirección, etc.)
 - PacientesInfo: contiene datos administrativos (obra social, fecha de alta, estado)
 - ActividadesBasicas y ActividadesDetalle: información sobre servicios ofrecidos
 - Turnos: turnos asignados a pacientes para diferentes actividades
 - LogCambios: tabla de auditoría de cambios

Separación de datos sensibles y operativos (partición vertical).

Gestión de **usuarios y roles** (para seguridad y permisos).

- 3) Implementar un **trigger** que registre en LogCambios cualquier modificación en nombre_completo o email en la tabla PacientesDatos.
- 4) Insertar al menos 5 pacientes, 4 actividades y 3 turnos de ejemplo.
- 5) Crear los siguientes **usuarios con permisos específicos**:

- admin: acceso total

- profesional: puede consultar los datos operativos de los pacientes, las actividades y registrar turnos
 - recepcion: puede registrar nuevos turnos y consultar los datos operativos
- 6) Probar los accesos de cada usuario conectándose con sus credenciales y verificando las restricciones.
 - 7) Simular un cambio de correo y nombre para un paciente y verificar que se haya registrado en el LogCambios.

2. PARTICIÓN VERTICAL

a. Crear base de datos y usarla

```
DROP DATABASE IF EXISTS InstitutoMedico;  
CREATE DATABASE InstitutoMedico;  
USE InstitutoMedico;
```

b. Tabla Pacientes → Partición vertical

Dividimos en dos tablas:

- **PacientesDatos** (sensibles):

```
CREATE TABLE PacientesDatos (  
    id_paciente INT PRIMARY KEY,  
    dni VARCHAR(20),  
    nombre_completo VARCHAR(100),  
    direccion VARCHAR(100),  
    telefono VARCHAR(20),  
    email VARCHAR(100)  
);
```

c. Tabla Pacientes Info

- **PacientesInfo** (operativa):

```
CREATE TABLE PacientesInfo (  
    id_paciente INT PRIMARY KEY,  
    fecha_alta DATE,  
    obra_social VARCHAR(100),  
    estado VARCHAR(20)  
);
```

d. Tabla Actividades del Instituto → Partición vertical

Por ejemplo:

1) ActividadesBasicas:

```
CREATE TABLE ActividadesBasicas (  
    id_actividad INT PRIMARY KEY,  
    nombre_actividad VARCHAR(100),  
    tipo VARCHAR(50)  
);
```

2) ActividadesDetalles:

```
CREATE TABLE ActividadesDetalle (  
    id_actividad INT PRIMARY KEY,  
    duracion INT,  
    costo DECIMAL(10,2),  
    responsable VARCHAR(100)  
);
```

3) LogCambios

```
CREATE TABLE LogCambios (  
    id_log INT AUTO_INCREMENT PRIMARY KEY,  
    tabla_afectada VARCHAR(50),  
    id_afectado INT,  
    campo_modificado VARCHAR(50),  
    valor_anterior VARCHAR(255),  
    valor_nuevo VARCHAR(255),  
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

4) Tabla de turnos

```
CREATE TABLE Turnos (  
    id_turno INT AUTO_INCREMENT PRIMARY KEY,  
    id_paciente INT,  
    id_actividad INT,  
    fecha_turno DATE,
```

```
hora TIME,  
estado VARCHAR(20),  
FOREIGN KEY (id_paciente) REFERENCES PacientesDatos(id_paciente),  
FOREIGN KEY (id_actividad) REFERENCES  
ActividadesBasicas(id_actividad)  
);
```

e. TRIGGER PARA AUDITAR CAMBIOS

Trigger de ejemplo (sobre cambio de nombre o email en PacientesDatos)

```
DELIMITER //  
  
CREATE TRIGGER tr_log_cambios_pacientes  
BEFORE UPDATE ON PacientesDatos  
FOR EACH ROW  
BEGIN  
    IF OLD.nombre_completo <> NEW.nombre_completo THEN  
        INSERT INTO LogCambios (tabla_afectada, id_afectado,  
campo_modificado, valor_anterior, valor_nuevo)  
VALUES ('PacientesDatos', OLD.id_paciente, 'nombre_completo',  
OLD.nombre_completo, NEW.nombre_completo);  
    END IF;  
  
    IF OLD.email <> NEW.email THEN  
        INSERT INTO LogCambios (tabla_afectada, id_afectado,  
campo_modificado, valor_anterior, valor_nuevo)  
VALUES ('PacientesDatos', OLD.id_paciente, 'email', OLD.email,  
NEW.email);  
    END IF;  
END;  
  
//  
  
DELIMITER ;
```

f. Insertar datos de ejemplo

```
INSERT INTO PacientesDatos VALUES  
(1, '12345678', 'Ana López', 'Calle Falsa 123', '1122334455',  
'ana.lopez@email.com'),  
(2, '23456789', 'Carlos Méndez', 'Av. San Martín 456', '1133445566',  
'carlos.mendez@email.com'),
```

```
(3, '34567890', 'Laura Torres', 'Rivadavia 789', '1144556677',  
'laura.torres@email.com'),  
(4, '45678901', 'Julián Suárez', 'Belgrano 321', '1155667788',  
'julian.suarez@email.com'),  
(5, '56789012', 'María Fernanda Díaz', 'Sarmiento 111', '1166778899',  
'm.fernanda@email.com');
```

```
INSERT INTO PacientesInfo VALUES  
(1, '2023-10-01', 'OSDE', 'activo'),  
(2, '2023-09-15', 'PAMI', 'activo'),  
(3, '2024-01-20', 'OSDE', 'inactivo'),  
(4, '2024-03-12', 'Swiss Medical', 'activo'),  
(5, '2024-06-05', 'Galeno', 'activo');
```

```
INSERT INTO ActividadesBasicas VALUES  
(1, 'Consulta Clínica', 'Consulta'),  
(2, 'Estudios de Laboratorio', 'Estudio'),  
(3, 'Psicoterapia', 'Consulta'),  
(4, 'Control Cardiológico', 'Especialidad');
```

```
INSERT INTO ActividadesDetalle VALUES  
(1, 30, 2500.00, 'Dra. Fernández'),  
(2, 20, 3500.00, 'Lic. Romero'),  
(3, 45, 4500.00, 'Dra. Navarro'),  
(4, 30, 6000.00, 'Dr. Quiroga');
```

```
INSERT INTO Turnos (id_paciente, id_actividad, fecha_turno, hora,  
estado) VALUES  
(2, 2, '2024-07-01', '09:30:00', 'confirmado'),  
(3, 3, '2024-07-02', '10:00:00', 'pendiente'),  
(4, 4, '2024-07-03', '08:45:00', 'cancelado');
```

f. USUARIOS Y PERMISOS EN MySQL

Crear usuarios con permisos diferentes

-- Eliminamos usuarios si existen para evitar errores

```
DROP USER IF EXISTS 'admin'@'localhost';
```

```
DROP USER IF EXISTS 'profesional'@'localhost';
```

```
DROP USER IF EXISTS 'recepcion'@'localhost';
```

-- Creamos los usuarios

```
CREATE USER 'admin'@'localhost' IDENTIFIED BY 'adminpass';
```

```
CREATE USER 'profesional'@'localhost' IDENTIFIED BY 'clave123';
```

```
CREATE USER 'recepcion'@'localhost' IDENTIFIED BY 'recepass';
```

-- Asignar permisos

-- Admin: todos los permisos

```
GRANT ALL PRIVILEGES ON ConsultorioMédicoC2.* TO 'admin'@'localhost';
```

-- Profesional: puede ver info operativa, actividades y gestionar turnos

```
GRANT SELECT ON ConsultorioMédicoC2.PacientesInfo TO  
'profesional'@'localhost';
```

```
GRANT SELECT ON ConsultorioMédicoC2.ActividadesBasicas TO  
'profesional'@'localhost';
```

```
GRANT SELECT, INSERT, UPDATE ON ConsultorioMédicoC2.Turnos TO  
'profesional'@'localhost';
```

-- Recepción: puede ver info operativa y gestionar turnos

```
GRANT SELECT, INSERT ON ConsultorioMédicoC2.PacientesInfo TO  
'recepcion'@'localhost';
```

```
GRANT SELECT, INSERT ON ConsultorioMédicoC2.Turnos TO  
'recepcion'@'localhost';
```

-- Aplicar cambios

```
FLUSH PRIVILEGES;
```

g. Operatividad

Se actualiza un paciente y se graba el cambio en LogCambios.

Un profesional puede ver la tabla PacientesInfo, pero no PacientesDatos (te dará error).

Un recepcionista puede cargar turnos.

Un admin puede hacer todo.

h. Simulación para activar el Trigger (actualizar nombre y mail)

```
UPDATE PacientesDatos
```

```
SET nombre_completo = 'Carlos Andrés Méndez',
```

```
email = 'c.mendez@email.com'
```

```
WHERE id_paciente = 2;
```

i. Consultar log para verificar la auditoría

```
SELECT * FROM LogCambios;
```

j. Probar distintos usuarios

Conectarse como admin (tiene todos los permisos)

```
-- mysql -u admin -p
```

Te pedirá la contraseña: adminpass

Conectarse como profesional (solo algunos permisos)

```
-- mysql -u profesional -p
```

Contraseña: clave123

Conectarse como recepcionista

```
-- mysql -u recepcion -p
```

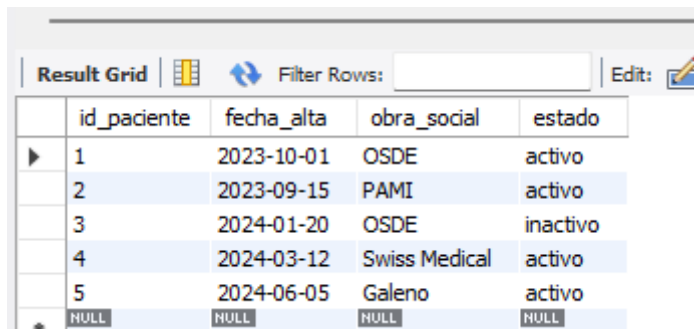
Contraseña: receppass

-- Una vez dentro de MySQL para usar la base de datos

USE ConsultorioMédicoC2;

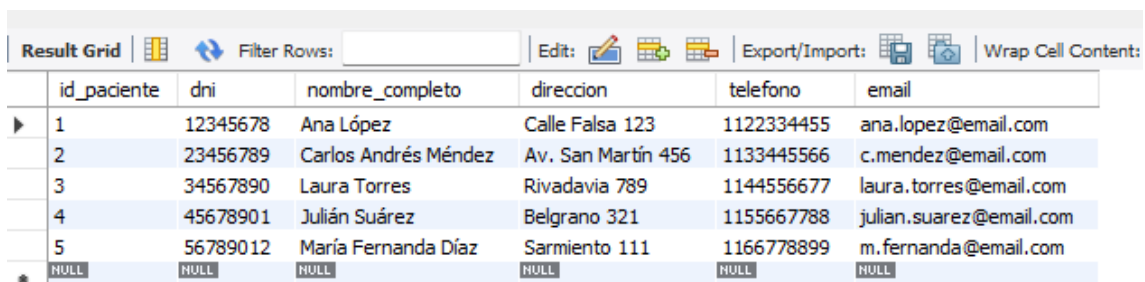
-- Con profesional podés probar

SELECT * FROM PacientesInfo; -- Debe funcionar



	id_paciente	fecha_alta	obra_social	estado
▶	1	2023-10-01	OSDE	activo
	2	2023-09-15	PAMI	activo
	3	2024-01-20	OSDE	inactivo
	4	2024-03-12	Swiss Medical	activo
	5	2024-06-05	Galeno	activo
*	NULL	NULL	NULL	NULL

SELECT * FROM PacientesDatos; -- Debe dar error (no tiene permiso)



	id_paciente	dni	nombre_completo	direccion	telefono	email
▶	1	12345678	Ana López	Calle Falsa 123	1122334455	ana.lopez@email.com
	2	23456789	Carlos Andrés Méndez	Av. San Martín 456	1133445566	c.mendez@email.com
	3	34567890	Laura Torres	Rivadavia 789	1144556677	laura.torres@email.com
	4	45678901	Julián Suárez	Belgrano 321	1155667788	julian.suarez@email.com
	5	56789012	María Fernanda Díaz	Sarmiento 111	1166778899	m.fernanda@email.com
*	NULL	NULL	NULL	NULL	NULL	NULL

INSERT INTO Turnos (id_paciente, id_actividad, fecha_turno, hora, estado)

VALUES (1, 1, '2025-01-01', '10:00:00', 'pendiente'); -- Debe funcionar

-- Con recepcionista

SELECT * FROM PacientesInfo; -- Permite consultar

Result Grid				
Filter Rows:				
Edit:				
	id_paciente	fecha_alta	obra_social	estado
▶	1	2023-10-01	OSDE	activo
	2	2023-09-15	PAMI	activo
	3	2024-01-20	OSDE	inactivo
	4	2024-03-12	Swiss Medical	activo
	5	2024-06-05	Galeno	activo
✱	NULL	NULL	NULL	NULL

INSERT INTO Turnos (id_paciente, id_actividad, fecha_turno, hora, estado)

VALUES (3, 2, '2025-01-02', '11:00:00', 'confirmado'); -- Permite insertar

Script completo

1- Creamos la DB y tablas

-- Limpiar si ya existe

DROP DATABASE IF EXISTS InstitutoMedico;

CREATE DATABASE InstitutoMedico;

USE InstitutoMedico;

-- Pacientes (partición vertical)

```
CREATE TABLE PacientesDatos (
    id_paciente INT PRIMARY KEY,
    dni VARCHAR(20),
    nombre_completo VARCHAR(100),
    direccion VARCHAR(100),
    telefono VARCHAR(20),
    email VARCHAR(100)
);
```

```
CREATE TABLE PacientesInfo (
    id_paciente INT PRIMARY KEY,
    fecha_alta DATE,
```



```
obra_social VARCHAR(100),  
estado VARCHAR(20)  
);
```

```
-- Actividades (partición vertical)  
CREATE TABLE ActividadesBasicas (  
    id_actividad INT PRIMARY KEY,  
    nombre_actividad VARCHAR(100),  
    tipo VARCHAR(50)  
);
```

```
CREATE TABLE ActividadesDetalle (  
    id_actividad INT PRIMARY KEY,  
    duracion INT,  
    costo DECIMAL(10,2),  
    responsable VARCHAR(100)  
);
```

```
-- Log de cambios  
CREATE TABLE LogCambios (  
    id_log INT AUTO_INCREMENT PRIMARY KEY,  
    tabla_afectada VARCHAR(50),  
    id_afectado INT,  
    campo_modificado VARCHAR(50),  
    valor_anterior VARCHAR(255),  
    valor_nuevo VARCHAR(255),  
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE Turnos (  
    id_turno INT AUTO_INCREMENT PRIMARY KEY,  
    id_paciente INT,  
    id_actividad INT,  
    fecha_turno DATE,  
    hora TIME,  
    estado VARCHAR(20)  
);
```

```
-- Permisos (si no lo hiciste antes)  
GRANT SELECT, INSERT ON InstitutoMedico.Turnos TO  
'repcion'@'localhost';  
GRANT SELECT, INSERT ON InstitutoMedico.Turnos TO  
'profesional'@'localhost';
```

```
-- Cargar turnos
```

```
INSERT INTO Turnos (id_paciente, id_actividad, fecha_turno, hora,
estado) VALUES
(2, 2, '2024-07-01', '09:30:00', 'confirmado'),
(3, 3, '2024-07-02', '10:00:00', 'pendiente'),
(4, 4, '2024-07-03', '08:45:00', 'cancelado');
```

2- TRIGGER PARA AUDITAR CAMBIOS EN PACIENTES

DELIMITER //

```
CREATE TRIGGER tr_log_cambios_pacientes
BEFORE UPDATE ON PacientesDatos
FOR EACH ROW
BEGIN
    IF OLD.nombre_completo <> NEW.nombre_completo THEN
        INSERT INTO LogCambios (tabla_afectada, id_afectado,
campo_modificado, valor_anterior, valor_nuevo)
VALUES ('PacientesDatos', OLD.id_paciente, 'nombre_completo',
OLD.nombre_completo, NEW.nombre_completo);
    END IF;

    IF OLD.email <> NEW.email THEN
        INSERT INTO LogCambios (tabla_afectada, id_afectado,
campo_modificado, valor_anterior, valor_nuevo)
VALUES ('PacientesDatos', OLD.id_paciente, 'email', OLD.email,
NEW.email);
    END IF;
END;
//

DELIMITER ;
```

3- INSERTAR DATOS DE PRUEBA

```
-- Paciente de ejemplo
INSERT INTO PacientesDatos VALUES
```

```
(1, '12345678', 'Ana López', 'Calle Falsa 123', '1122334455',  
'ana.lopez@email.com'),  
(2, '23456789', 'Carlos Méndez', 'Av. San Martín 456', '1133445566',  
'carlos.mendez@email.com'),  
(3, '34567890', 'Laura Torres', 'Rivadavia 789', '1144556677',  
'laura.torres@email.com'),  
(4, '45678901', 'Julián Suárez', 'Belgrano 321', '1155667788',  
'julian.suarez@email.com'),  
(5, '56789012', 'María Fernanda Díaz', 'Sarmiento 111', '1166778899',  
'm.fernanda@email.com');
```

INSERT INTO PacientesInfo VALUES

```
(1, '2023-10-01', 'OSDE', 'activo'),
(2, '2023-09-15', 'PAMI', 'activo'),
(3, '2024-01-20', 'OSDE', 'inactivo'),
(4, '2024-03-12', 'Swiss Medical', 'activo'),
(5, '2024-06-05', 'Galeno', 'activo');
```

-- Actividad de ejemplo

INSERT INTO ActividadesBasicas VALUES

(1, 'Consulta Clínica', 'Consulta'),
(2, 'Estudios de Laboratorio', 'Estudio'),
(3, 'Psicoterapia', 'Consulta'),
(4, 'Control Cardiológico', 'Especialidad');

INSERT INTO ActividadesDetalle VALUES

```
(1, 30, 2500.00, 'Dra. Fernández'),  
(2, 20, 3500.00, 'Lic. Romero'),  
(3, 45, 4500.00, 'Dra. Navarro'),  
(4, 30, 6000.00, 'Dr. Quiroga');
```

4- PRUEBA DEL TRIGGER

-- Cambiamos nombre y mail para activar el trigger

UPDATE PacientesDatos

```
SET nombre_completo = 'Ana María López',
```

```
email = 'ana.mlopez@email.com'
```

```
WHERE id_paciente = 1;
```

```
-- Ver el log de cambios
```

```
SELECT * FROM LogCambios;
```

[illegible]

5- SIMULACION DE CAMBIO EN DATOS (ACTIVA TRIGGER)

-- Cambiamos nombre y mail de un paciente para generar entradas en el log

```
UPDATE PacientesDatos
SET nombre_completo = 'Carlos Andrés Méndez',
    email = 'c.mendez@email.com'
WHERE id_paciente = 2;
```

Verificamos que se haya registrado el cambio

```
SELECT * FROM LogCambios;
```

6- CREAR USUARIOS Y ASIGNAR ROLES

-- Crear usuarios

```
CREATE USER 'admin'@'localhost' IDENTIFIED BY 'adminpass';
CREATE USER 'profesional'@'localhost' IDENTIFIED BY 'clave123';
CREATE USER 'repcion'@'localhost' IDENTIFIED BY 'receppass';
```

-- Permisos para admin

```
GRANT ALL PRIVILEGES ON InstitutoMedico.* TO 'admin'@'localhost';
```

-- Permisos para profesional

```
GRANT SELECT ON InstitutoMedico.PacientesInfo TO
'profesional'@'localhost';
GRANT SELECT ON InstitutoMedico.ActividadesBasicas TO
'profesional'@'localhost';
```

-- Permisos para recepcionista

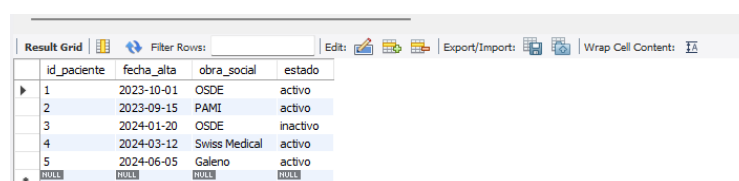
```
GRANT SELECT, INSERT ON InstitutoMedico.PacientesInfo TO
'repcion'@'localhost';
```

-- Aplicar cambios

```
FLUSH PRIVILEGES;
```

-- Para ver si funcionó

```
SELECT * FROM PacientesInfo;
```



	id_paciente	fecha_alta	obra_social	estado
▶	1	2023-10-01	OSDE	activo
	2	2023-09-15	PAMI	activo
	3	2024-01-20	OSDE	inactivo
	4	2024-03-12	Swiss Medical	activo
	5	2024-06-05	Galeno	activo
*	TOTAL	00000	00000	00000

SELECT * FROM PacientesDatos; -- acceso no concedido salvo que tu conexión sea como doctor o recepcionista

Result Grid						
		Filter Rows:	Edit:		Export/Import:	Wrap Cell
	id_paciente	dni	nombre_completo	direccion	telefono	email
▶	1	12345678	Ana López	Calle Falsa 123	1122334455	ana.lopez@email.com
	2	23456789	Carlos Andrés Méndez	Av. San Martín 456	1133445566	c.mendez@email.com
	3	34567890	Laura Torres	Rivadavia 789	1144556677	laura.torres@email.com
	4	45678901	Julián Suárez	Belgrano 321	1155667788	julian.suarez@email.com
	5	56789012	María Fernanda Díaz	Sarmiento 111	1166778899	m.fernanda@email.com
✱	NULL	NULL	NULL	NULL	NULL	NULL

-- verificamos los servicios

SHOW GRANTS FOR 'profesional'@'localhost';

Result Grid	
Filter Rows:	
Export:	
Wrap Cell Content:	
Grants for profesional@localhost	
▶	GRANT USAGE ON *.* TO `profesional`@`local...
	GRANT SELECT ON `institutomedico`.`activida...
	GRANT SELECT ON `institutomedico`.`paciente...
	GRANT SELECT, INSERT ON `institutomedico`....

Si todos los permisos están bien, el resultado será

GRANT USAGE ON *.* TO 'profesional'@'localhost'

GRANT SELECT ON `InstitutoMedico`.`PacientesInfo` TO 'profesional'@'localhost'

GRANT SELECT ON `InstitutoMedico`.`ActividadesBasicas` TO 'profesional'@'localhost'

GRANT SELECT, INSERT ON `InstitutoMedico`.`Turnos` TO 'profesional'@'localhost'